INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

**Département GM**

# Cahier des charges

*Simulation et optimisation de trajectoire d'un satellite autonome*

**Encadrant:**
**M. A. OSMANI**

**Présenté par:**
**Mouad SHERADJ**
**DRISSI**
**Amr ZAKI SALIH**

---

**Année universitaire: 2025-2026**

# Table des matières

1 Contexte . . . . . 2
2 Enjeu . . . . . 2
3 Besoins . . . . . 3
3.1 Besoins fonctionnels . . . . . 3
3.2 Besoins non fonctionnels . . . . . 3
4 Organisation . . . . . 3
5 Critères de réussites . . . . . 5
6 Modélisation . . . . . 6
7 Livrable . . . . . 7
8 Etat de l'art . . . . . 7

# Introduction

Le présent document constitue le cahier des charges du projet de simulation et optimisation de trajectoire d'un satellite autonome. Ce projet a pour objectif de concevoir et développer un programme complet en langage Java intégrant trois dimensions essentielles : la communication réseau, une interface graphique utilisateur et une gestion de base de données.

Le système doit modéliser un environnement spatial composé de plusieurs astres, de satellites et de contraintes physiques (obstacles, météorites, interférences). L'utilisateur pourra définir un point de départ et une destination, et le programme déterminera la trajectoire optimale à suivre en tenant compte de ces contraintes. Les satellites échangeront entre eux des données collectées par des antennes, formant un réseau interconnecté et coopératif.

Enfin, un aspect d'intelligence artificielle sera ajouté afin d'optimiser la trajectoire selon les conditions dynamiques de l'environnement spatial. L'objectif est d'analyser en continu les conditions de l'environnement spatial, d'anticiper les obstacles et d'adapter dynamiquement la trajectoire.

# 1 Contexte

Ce projet s'inscrit dans le cadre de la Programmation Orientée Objet Avancée, qui a pour objectif de mettre en pratique les principes de conception et de développement vus en cours. Le thème retenu s'inspire d'un projet semestriel plus large portant sur l'analyse de données satellitaires, et en particulier sur l'étude du lien entre assèchement du sol et hauteur du sol en Europe. Pour adapter ce sujet à un contexte applicatif orienté objet, le choix de développer une simulation centrée sur les interactions entre satellites et entre satellites et utilisateurs a été réalisé. Le système devra permettre de déterminer la trajectoire optimale d'un satellite pour atteindre une position cible, tout en collectant des informations issues des autres satellites présents dans l'environnement simulé. Ce projet vise donc à concilier l'apprentissage des concepts de la Programmation Orienté Objet, en particulier en Java, avec une problématique concrète et inspirée du domaine spatial.

# 2 Enjeu

L'enjeu de ce projet de simulation vise dans un premier temps à mettre en œuvre de manière concrète les concepts de programmation orienté objet tels que la communication réseau, les interfaces graphiques utilisateur ainsi que la modularité, l'héritage et l'encapsulation dans le développement d'une application concrète. Ce projet inclut donc des notions de programmation orientée objet étudiées lors de ce semestre ainsi que du semestre dernier. L'enjeu porte aussi sur le défi d'algorithmie et de simulation qu'impose un tel projet puisqu'il s'agit de déterminer des trajectoires optimales pour les satellites tout en intégrant des échanges d'informations entre ces derniers. Il s'inspire ainsi de problématiques considérées comme réelles rencontrées dans le domaine spatial, notamment en matière de communication. La réussite du projet démontrera la capacité de la programmation orientée objet à répondre à des problématiques complexes.

# 3 Besoins

## 3.1 Besoins fonctionnels

Le programme doit modéliser un espace composé de plusieurs astres (planètes, lunes, ...), satellites, et contraintes physiques dynamiques telles que les obstacles (débris spatiaux), météorites, et interférences (champs gravitationnels). Toute trajectoire calculée doit obligatoirement prendre en compte ces éléments pour éviter les collisions ou les déviations non désirées. Le programme doit permettre à l'utilisateur de définir un point de départ et une destination via l'interface. Le programme doit alors calculer une trajectoire optimale, en intégrant des contraintes physiques (gravité, vitesse, énergie). Les calculs doivent être conformes aux lois de la mécanique orbitale. Les satellites doivent former un réseau interconnecté et coopératif, échangeant des données collectées par des antennes. Ces échanges doivent être gérés via des protocoles de communication réseau. Les données collectées doivent être nettoyées, analysées pour optimiser les trajectoires, et utilisées pour estimer la durée des parcours. Le système doit inclure une gestion de base de données pour stocker les données collectées (positions, trajectoires historiques, analyses). Une intelligence artificielle (IA) traitera ces informations en temps réel afin d'améliorer la précision des prédictions et des calculs de trajectoire. Le modèle d'IA devra également prédire et proposer des trajectoires dynamiques selon les données collectées simultanément au déplacement du satellite concerné. Ces trajectoires alternatives seront ensuite proposées à l'utilisateur comme solutions possibles en cas de changement des conditions physiques ou environnementales.

## 3.2 Besoins non fonctionnels

Le développement doit être exclusivement en Java structuré en architecture orientée objet et le code doit être documenté (commentaires, Javadoc) pour faciliter la compréhension, la maintenance, et les évolutions futures.

# 4 Organisation

Le projet sera réalisé par deux étudiants de Génie Mathématique qui sont ZAKI SALIH Amr ainsi que SHERADJ DRISSI Mouad. Sa réalisation se déroulera de octobre à décembre. Etant donné un grand nombre de contraintes phyiques, le travail sera organisé sous la forme d'une méthode AGILE. La méthode Agile est une façon de gérer le projet qui mettra l'accent sur la souplesse, la collaboration et les adaptations rapides plutôt que sur une planification rigide dès le départ.

Le planning prévisionnel s'étend sur 8 semaines n'ayant pas de date de rendu fixe, il est nécessaire de réaliser le projet avant son échéance réelle. Le planning est donc de la sorte :

- Semaine 1:
  - Définition du cahier des charges

- Définition du Use Case

- Semaine 2 à semaine 3 :
  - Conception et modélisation :
    - Réalisation du diagramme de classes
    - Réalisation du diagramme d'objets
    - Réalisation du diagramme de séquences

- Semaine 3 à Semaine 7 :
  - Développement en Java:
    - Sprint 1 : Conception de la base d'un satellite dans le modèle statique
    - Sprint 2 : Ajout des interactions satellites-satellites
    - Sprint 3 : Ajout des interactions Satellites-utilisateur
    - Sprint 4 : Ajout des contraintes physiques à l'environnement du modèle statique
    - Sprint 5 : Interface graphique
    - Sprint 6 : Tests unitaires, validation du code

- Semaine 7 à Semaine 8 :
  - Validation Finale

  - Documentation : Rapport, documentation java

Si le temps le permet, une application IA sera intégrée au programme après le semaine 8. Elle consistera en l'ajout d'un modèle dynamique qui consistera à l'ajout de contraintes dynamiques dans un système dit donc dynamique. Il sera offert à l'utilisateur le choix d'utiliser soit le système dynamique ou soit le système statique.

Le diagramme de Gantt ci-contre représente le possible déroulé de la réalisation du projet :

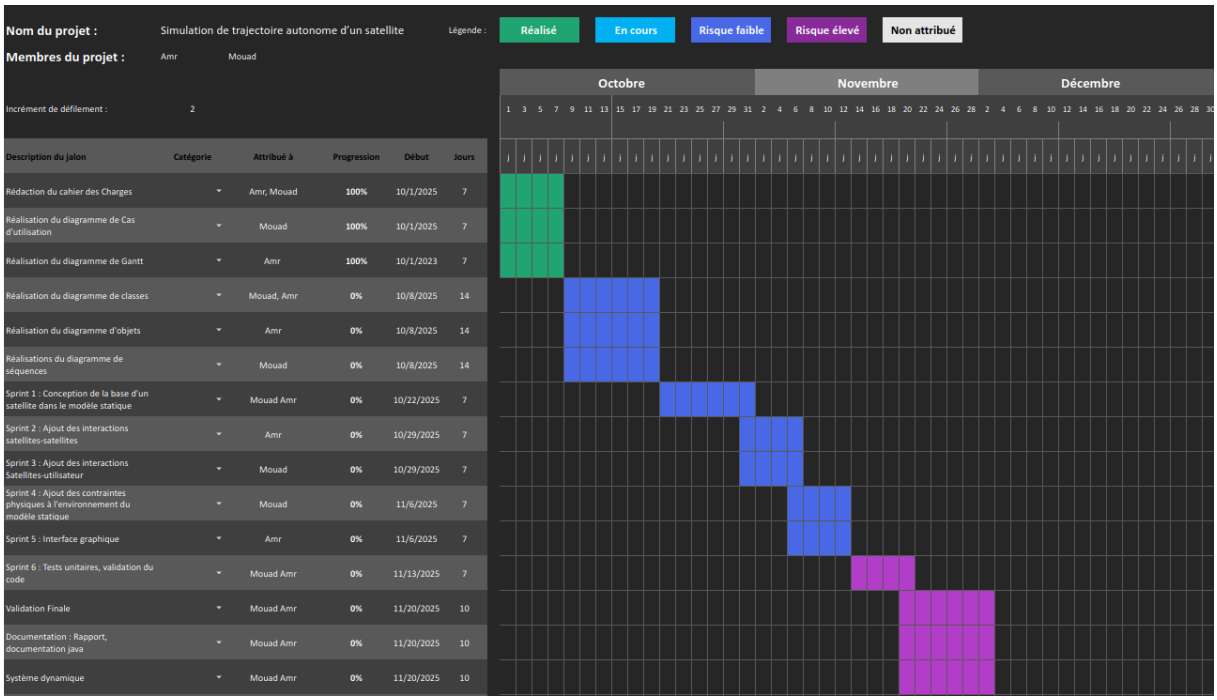



Figure 1.1 – *Diagramme de Gantt*

# 5 Critères de réussites

Le projet sera considéré comme réussi s'il respecte les objectifs fixés et les délais de réalisation. Le système devra comprendre :

- une interface utilisateur permettant de choisir les destinations ;
- une interface graphique interactive représentant le système spatial ;
- des calculs de trajectoire précis et conformes aux lois physiques ;
- un traitement efficace des données collectées pour l'optimisation des trajectoires ;
- une IA capable de proposer des trajectoires alternatives dynamiques en fonction des conditions en temps réel.

Le programme final devra être stable, compilable sans erreur et offrir une interface claire. La documentation devra être complète et claire et le projet devra être livré dans les délais prévus.

# 6 Modélisation

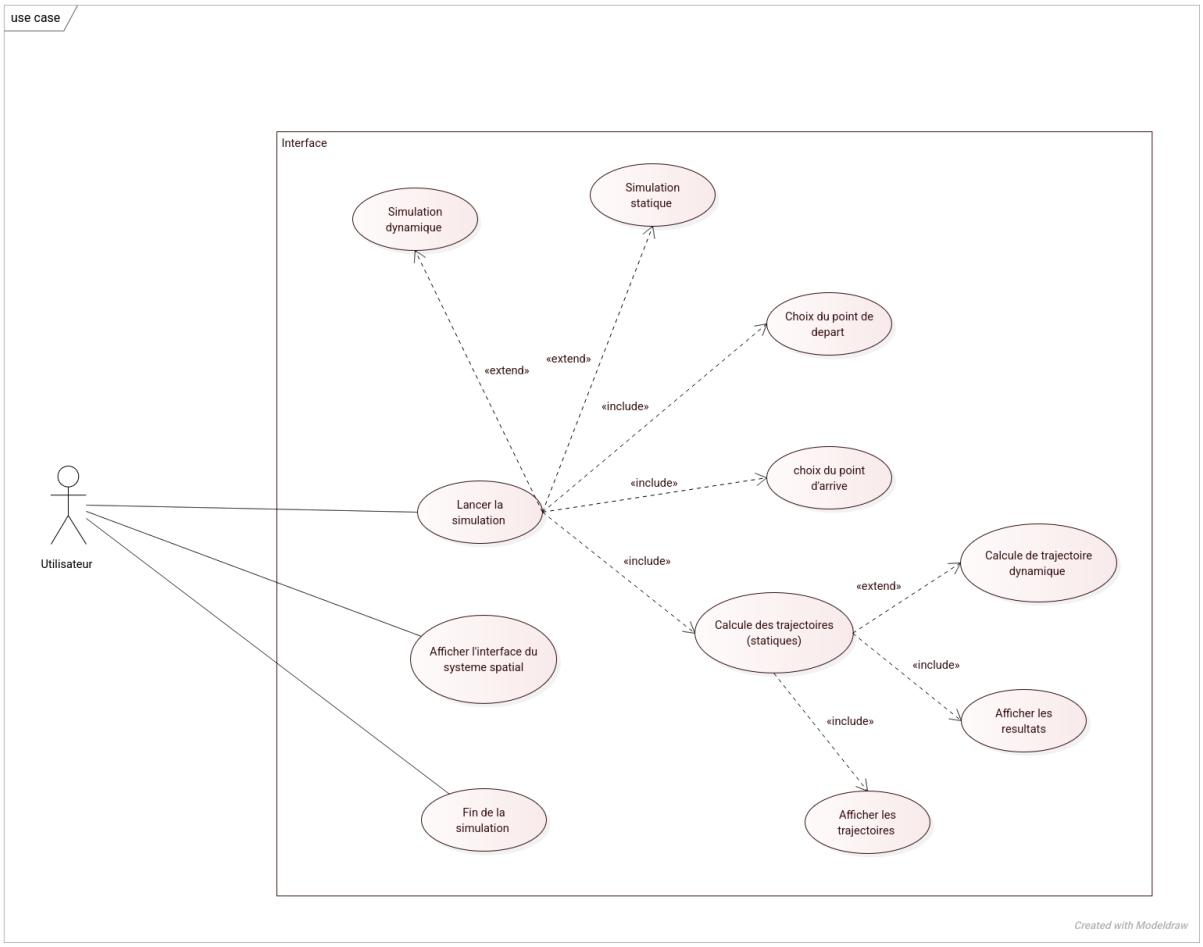



Figure 1.2 – *Diagramme de cas d'utilisation*

Ce diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur et le système. L'utilisateur commence par afficher l'interface du système spatial, puis lance la simulation.

Avant le calcul, il choisit un point de départ et un point d'arrivée. Le système effectue ensuite le calcul des trajectoires statiques, qui peut être étendu à une simulation dynamique. Dans ce cas, un modèle d'intelligence artificielle prédit et propose des trajectoires en temps réel selon les données collectées pendant le déplacement du satellite, tout en suggérant des solutions optimisées à l'utilisateur.

Enfin, le système affiche les trajectoires et les résultats obtenus, et l'utilisateur peut mettre fin à la simulation.

# 7 Livrable

Livrables attenuds :

- Code source du simulateur en Java
- Application fonctionnelle permettant la simulation des interactions satellites-utilisateur et satellites-satellites
- Documentation technique et guide d'utilisation (areadme)
- Diagrammes UML (cas d'utilisations, classes, séquences, objets)
- Rapport final détaillant la conception, la modélisation et les résultats issue des tests

# 8 Etat de l'art

- **Orekit** – https://github.com/CS-SI/Orekit
  Orekit est une bibliothèque Java open-source développée par CS Group, dédiée à la mécanique orbitale et à la modélisation des mouvements de satellites. Elle fournit des outils très complets pour la propagation orbitale, les systèmes de coordonnées, la gestion du temps, les manœuvres, la navigation GNSS ou encore le suivi de missions spatiales.
  - **Intérêt :** Fiabilité scientifique et précision des calculs orbitaux. Elle peut servir de moteur de physique pour tout projet simulant des trajectoires de satellites.
  - **Lacunes :** Orekit ne gère pas la simulation réseau entre satellites (échanges de données, liaisons inter-satellites, latences, bande passante) ni l'interaction directe avec un utilisateur.
  - **Apports possibles :** Intégrer Orekit comme base de calcul physique dans une architecture complète, en la couplant à un simulateur réseau et à une interface utilisateur interactive. Cela permettrait de combiner la précision orbitale d'Orekit avec des communications et des interactions temps réel.
- **Satellite-Network-Simulators** – https://github.com/jwwthu/Satellite-Network-Simulators
  Ce dépôt recense plusieurs simulateurs de réseaux de satellites open-source, en regroupant leurs caractéristiques, langages et objectifs. Il ne s'agit pas d'un simulateur unique, mais d'une collection de projets comparatifs.
  - **Intérêt :** Fournit une vue d'ensemble des principaux outils open-source pour les réseaux satellitaires, utile pour choisir les technologies existantes et comprendre les approches courantes.
  - **Lacunes :** Ce dépôt ne contient pas de code exploitable directement ni d'environnement intégré. Il se limite à une liste et une comparaison.
  - **Apports possibles :** Créer un environnement unifié inspiré de cette collection, combinant simulation orbitale et réseau, plutôt qu'un simple catalogue. Le projet pourrait en être une synthèse opérationnelle.
- **Large-Scale Satellite Network Simulator (LSNS)** – https://github.com/infonetlijian/Large-Scale-Satellite-Network-Simulator-LSNS
  LSNS est un simulateur Java orienté « grands réseaux de satellites » (constellations,

interconnexions et routage). Il dérive du simulateur ONE et vise la simulation à grande échelle de communications spatiales (Delay Tolerant Networks, etc.).
  - **Intérêt :** Permet de modéliser la topologie et les communications d'une grande constellation. Outil performant pour tester des protocoles de routage ou d'accès.
  - **Lacunes :** Simulation réseau uniquement — la position des satellites est simplifiée et la mécanique orbitale réelle n'est pas intégrée. L'interactivité et le rendu visuel sont également limités.
  - **Apports possibles :** Coupler LSNS à un moteur physique précis (par exemple Orekit) afin que les liaisons réseau dépendent de la position réelle des satellites, et ajouter une interface utilisateur permettant de visualiser et interagir avec le système.
- **SatTrack** – https://github.com/snowclipsed/SatTrack
  SatTrack est une application Java qui permet de suivre en temps réel la position de satellites, grâce à une interface graphique connectée à des données d'orbite (TLE). Elle se concentre sur la visualisation et le suivi des objets spatiaux.
  - **Intérêt :** Fournit une interface claire et réactive pour afficher la position des satellites en direct sur une carte, ce qui est utile pour la visualisation d'orbites.
  - **Lacunes :** Pas de simulation de communication ni d'interaction entre satellites ; uniquement du tracking. Aucune prise en compte de scénarios utilisateurs.
  - **Apports possibles :** Reprendre la logique d'affichage temps réel de SatTrack pour l'appliquer à une simulation interactive où l'utilisateur peut influencer les satellites (commandes, transferts de données, manœuvres, etc.).
- **OrbitSimulation** – https://github.com/Brusa99/OrbitSimulation
  Projet Python universitaire simulant les trajectoires de satellites et l'évitement de collisions, avec un rendu simplifié des orbites et de la gravitation.
  - **Intérêt :** Simple à comprendre et à adapter pour tester rapidement des comportements orbitaux ou des interactions de base entre objets.
  - **Lacunes :** Approche très simplifiée de la physique (pas d'intégration complète des forces réelles), pas de gestion réseau ni d'interface utilisateur interactive.
  - **Apports possibles :** Intégrer des modèles plus réalistes (via Orekit), enrichir la simulation avec des échanges de données et une interface graphique pour visualiser les interactions satellites-satellites.
- **satellite-simulator (ltstrange)** – https://github.com/ltstrange/satellite-simulator
  Simulateur développé avec le moteur Bevy (Rust), intégrant un rendu visuel 3D ainsi que des modules de gestion d'énergie, de communication et de détection de collisions.
  - **Intérêt :** Offre une approche plus moderne et immersive de la simulation de satellites, combinant gestion de ressources et rendu 3D temps réel.
  - **Lacunes :** Projet expérimental avec peu de documentation, non interfacé avec des modèles orbitaux réalistes et sans gestion multi-satellite détaillée.
  - **Apports possibles :** S'inspirer de son rendu visuel pour concevoir une interface 3D interactive tout en y intégrant la précision orbitale d'Orekit et la gestion réseau d'un simulateur comme LSNS.